

Carrera de Especialización en Microelectrónica – FIUBA

Diseño Lógico

TP1: función computada por un AFD

Alumno: Veglia Lucas

Consigna: De un texto del idioma castellano de máximo 1.000 caracteres, se quiere reconocer cuántas veces aparece la palabra “casa”. Diseñar el AFD (cuántos registros de 1 bit se necesita, hallar la función de traducción $f: N \rightarrow N$ que calcula el traductor, hallar las funciones ω y δ . TIP: se considerará que solo pueden aparecer los siguientes símbolos en el texto: a, b, c, d, e, f, g, h, i, j, k, l, m, n, ñ, o, p, q, r, s, t, u, v, w, x, y, z, ‘ ’ (espacio en blanco para separar).

Un autómata finito determinístico (AFD) no puede contar sin la ayuda de un registro externo y si la entrada no está acotada. En este caso, la entrada sí está acotada. La única forma de que el AFD “cuenta” por sí solo, es asignando una función de traducción en la que distintos estados devuelvan distintos caracteres de salida.

Teniendo en cuenta que el alfabeto Σ posee 28 caracteres, y los textos a introducir pueden poseer de 0 a 1000 caracteres de longitud, la cantidad posible de textos de entrada es el resultado de:

$$\sum_{i=0}^{1000} 28^i$$

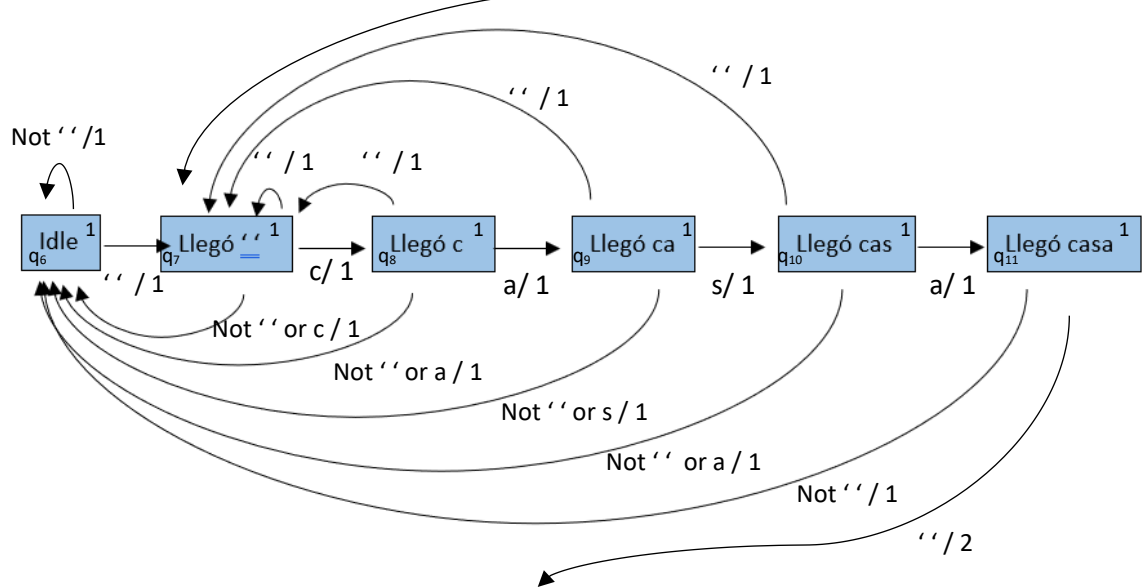
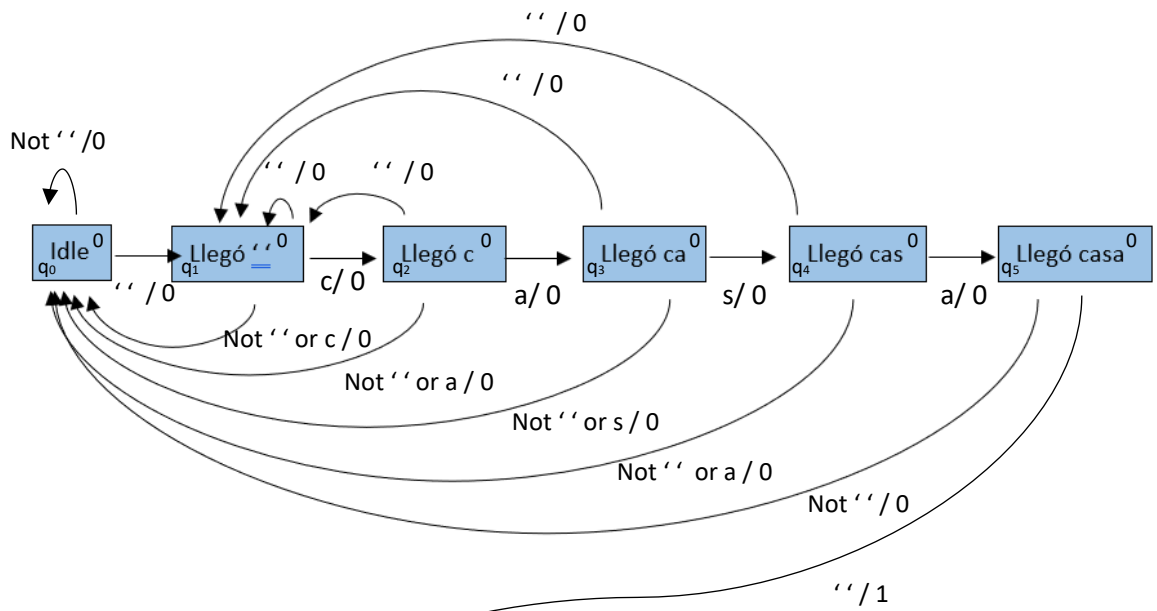
Sin embargo, la máxima ocurrencia de la secuencia de caracteres “casa’ ” en un texto de 1000 caracteres es de $1000/5 = 200$ veces (siendo ‘ ’ el carácter de espaciado).

$$\sum_{i=0}^{1000} 28^i$$

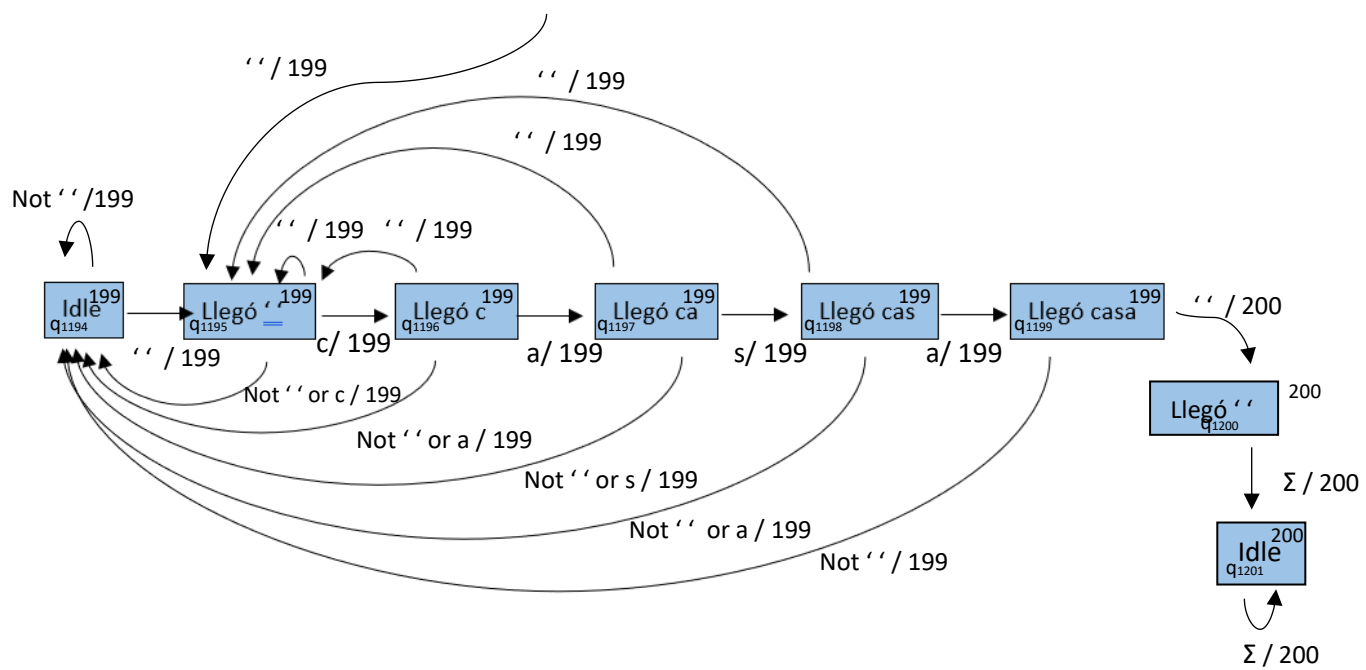
Por ende, la función a calcular por el traductor será una función que va de n entre 0 y $a f(n)$ entre 0 y 200, estando la función acotada. La tabla luciría de esta forma:

X	n	f(n)
‘ ’	1	0
a	2	0
aa	3	0
aab	4	0
...	...	...
aab....casa’ ^{‘0}		0
aab....casa’ ^{‘1}		1
aac....casa’ ^{‘1}		1
...	...	...
aaaz....casa’ ^{‘1} ...zzz	j	1
aab....casa’ ^{‘2} ...yzz	j+1	2
...	...	
Aaaa’ ‘casa’ ^{‘199}	k	199
Aaab’ ‘casa’ ^{‘199}	k+1	199
...	...	...
casa’ ^{‘200}	$\sum_{i=0}^{1000} 28^i$	200

La representación mediante un grafo se muestra a continuación:



... Conteos del 2 al 198



El AFD inicia en el estado q0 donde se encuentra inactivo (idle). Para transicionar al siguiente estado q1, debe recibir un carácter de espaciado ' ' como carácter de entrada. Esto es para poder separar sin ambigüedad la palabra casa, y que el AFD no cuente como ocurrencias palabras como "carcasa".

Para transicionar al estado q2, debe recibir el carácter 'c'. Para transicionar al estado q3, debe recibir el carácter 'a'. Para transicionar al estado q4, debe recibir el carácter 's'. Para transicionar al estado q5, debe recibir el carácter 'a'. En todos estos estados, si en las transiciones se recibe cualquier carácter distinto a los indicados, el AFD retrocederá al estado q0 o al estado q1 si lo que se recibió fue un carácter de espaciado ' '.

Para que el AFD cuente la primera ocurrencia de la palabra 'casa', se debe transicionar del estado q5 al estado q7, al recibir un carácter de espaciado ' '. Esto es para que el AFD no cuente como ocurrencias palabras que empiezan por casa como "casamiento". Al realizar la transición exitosamente, la función de traducción devolverá el valor "1", indicando que se ha contado la primera ocurrencia de la palabra casa. En todos los estados anteriores, la función de traducción devolvía el valor "0", al no haberse contado una ocurrencia correctamente.

Desde el estado q7, el ciclo de evaluación de la palabra casa se repite. Para transicionar a q8, se debe recibir 'c'. Para q9, se debe recibir 'a'. Para q10, se debe recibir 's' y para q11 se debe recibir 'a'. En cualquier punto si se recibe un carácter distinto al indicado, el AFD puede volver a q6 que es el nuevo estado idle, pero que mantiene la salida de '1', almacenando la cuenta de la primera ocurrencia de 'casa'.

Igual al primer caso, para que el AFD cuente la segunda ocurrencia de la palabra 'casa', debe transicionar del estado q11 al estado q13 mediante la aceptación del carácter ' '. Desde ese momento la salida de la función de traducción será '2'.

El flujo de evaluación y conteo se repetirá de manera similar a las descritas anteriormente, hasta llegar al estado q1200, estado en el que el AFD contabiliza 200 ocurrencias de la palabra casa, que es la máxima cantidad de veces posible que puede aparecer la palabra casa separada por espacios en un texto de 1000 caracteres. Cualquier otro carácter que se ingrese posteriormente en caso de que el texto supere los 1000 caracteres, el AFD pasa a un estado idle final, q1201, del cual no puede salir, finalizando así el conteo.

Este sistema cuenta con la desventaja de que no tiene forma de saber cuantos caracteres ya han ingresado. Por ende, puede darse el caso de que ingrese un texto mas largo, en el que la palabra 'casa' aparece luego de 1000 caracteres, y el contador indique una ocurrencia. La única forma de salvar esto es acotando la entrada forzando a que esta posea como máximo 1000 caracteres, como se planteó al principio.

Al poseer 1202 estados, serian necesarios $2^{11} = 2048$ -> 11 registros de 1 bit para poder diseñar este AFD. Suena mucho para una tarea tan sencilla, pero recordemos que al no poseer un contador externo, el AFD necesita estados distintos para llevar la cuenta.

La función de transición δ sería de la forma:

$\delta(q_i, s_i) = q_{i+1}$		
Estado actual q_i	Simbolo de entrada s_i	Estado siguiente q_{i+1}
q_0 (idle) ⁰	Not ' '	q_0 (idle) ⁰
q_0 (idle) ⁰	' '	q_1 (llegó ' ') ⁰
q_1 (llegó ' ') ⁰	Not ' ' or 'c'	q_0 (idle) ⁰
q_1 (llegó ' ') ⁰	' '	q_1 (llegó ' ') ⁰
q_1 (llegó ' ') ⁰	'c'	q_2 (llegó 'c ') ⁰
q_2 (llegó 'c') ⁰	Not ' ' or 'a'	q_0 (idle) ⁰
q_2 (llegó 'c') ⁰	' '	q_1 (llegó ' ') ⁰
q_2 (llegó 'c') ⁰	'a'	q_3 (llegó 'ca ') ⁰
q_3 (llegó 'ca') ⁰	Not ' ' or 's'	q_0 (idle) ⁰
q_3 (llegó 'ca') ⁰	' '	q_1 (llegó ' ') ⁰
q_3 (llegó 'ca') ⁰	's'	q_4 (llegó 'cas ') ⁰
q_4 (llegó 'cas') ⁰	Not ' ' or 's'	q_0 (idle) ⁰
q_4 (llegó 'cas') ⁰	' '	q_1 (llegó ' ') ⁰
q_4 (llegó 'cas') ⁰	'a'	q_5 (llegó 'casa ') ⁰
q_5 (llegó 'casa') ⁰	Not ' '	q_0 (idle) ⁰
q_5 (llegó 'casa') ⁰	' '	q_7 (llegó ' ') ¹
q_6 (idle) ¹	Not ' '	q_6 (idle) ¹
q_6 (idle) ¹	' '	q_7 (llegó ' ') ¹
q_7 (llegó ' ') ¹	Not ' ' or 'c'	q_6 (idle) ¹
q_7 (llegó ' ') ¹	' '	q_7 (llegó ' ') ¹
q_7 (llegó ' ') ¹	'c'	q_8 (llegó 'c ') ¹
q_8 (llegó 'c') ¹	Not ' ' or 'a'	q_6 (idle) ¹
q_8 (llegó 'c') ¹	' '	q_7 (llegó ' ') ¹
q_8 (llegó 'c') ¹	'a'	q_9 (llegó 'ca ') ¹
q_9 (llegó 'ca') ¹	Not ' ' or 's'	q_6 (idle) ¹
q_9 (llegó 'ca') ¹	' '	q_7 (llegó ' ') ¹
q_9 (llegó 'ca') ¹	's'	q_{10} (llegó 'cas ') ¹
q_{10} (llegó 'cas') ¹	Not ' ' or 's'	q_6 (idle) ¹
q_{10} (llegó 'cas') ¹	' '	q_7 (llegó ' ') ¹
q_{10} (llegó 'cas') ¹	'a'	q_{11} (llegó 'casa ') ¹
q_{11} (llegó 'casa') ¹	Not ' '	q_6 (idle) ¹
q_{11} (llegó 'casa') ¹	' '	q_{13} (llegó ' ') ²
...	...	...
q_{1199} (llegó 'casa') ¹⁹⁹	Not ' '	q_{1194} (idle) ¹⁹⁹
q_{1199} (llegó 'casa') ¹⁹⁹	' '	q_{1200} (llegó ' ') ²⁰⁰
q_{1200} (llegó ' ') ²⁰⁰	Cualquiera	q_{1201} (idle) ²⁰⁰
q_{1201} (idle) ²⁰⁰	Cualquiera	q_{1201} (idle) ²⁰⁰

La función de traducción ω sería de la forma:

$\omega(q) = y$	
Estado actual q	Símbolo de salida y
q0 (idle) ⁰	0
q1 (llegó ' ') ⁰	0
q2 (llegó 'c') ⁰	0
q3 (llegó 'ca') ⁰	0
q4 (llegó 'cas') ⁰	0
q5 (llegó 'casa') ⁰	0
q6 (idle) ¹	1
q7 (llegó ' ') ¹	1
q8 (llegó 'c') ¹	1
q9 (llegó 'ca') ¹	1
q10 (llegó 'cas') ¹	1
q11 (llegó 'casa') ¹	1
q12 (idle) ²	2
q13 (llegó ' ') ²	2
...	...
q1199 (llegó 'casa') ¹⁹⁹	199
q1200 (llegó ' ') ²⁰⁰	200
q1201(idle) ²⁰⁰	200

Quedando de esta manera definido el autómata finito determinístico A por la sextupla:

$A = \langle Q, \Sigma, \Delta, \delta, q_0, F \rangle$

$A = \langle \{q_0, q_1, q_2, \dots, q_{1199}, q_{1200}, q_{1201}\}, \{a, b, c, d, \dots, x, y, z, ' '\}, \{0, 1, 2, 3, \dots, 199, 200\}, \delta, q_0 \text{ (idle)}, F=Q \rangle$

(Con Q = conjunto finito de estados, Σ = alfabeto de entrada, Δ = alfabeto de salida, δ = función de transición, q_0 = estado inicial, F = conjunto de estados finales)